Apuntes del Sprint 16 · Cerrar un proyecto de IA: sesgos, demostrador, model card y memoria

La presentación del Sprint 16 es muy esquemática: estructura de carpetas, un par de pruebas y una lista de congelación. Estos apuntes **explican qué se pide en cada parte y por qué**, con ejemplos numéricos, para que el último esfuerzo no sea una carrera de última hora. Léelos antes de empezar el sprint y vuelve a ellos antes de la congelación. Este es **el sprint de la entrega**. A las **14:30 del 10/12** se congela el repositorio: lo que no esté subido **no cuenta**. Por eso la regla de oro es **ir cerrando cosas cada día**, no dejarlo todo para el final.

0. De qué va el sprint (en 1 minuto)

Un modelo que **solo vive en tu portátil no es un producto**. Este sprint convierte vuestro modelo en algo que **otra persona puede usar, entender, reproducir y cuestionar**.

Pieza	Para qué sirve	Para quién
Evaluación final y errores	Saber **cuánto vale** el modelo y **dónde falla**	El cliente y vosotros
Análisis de sesgos + mitigación	Comprobar que **no funciona peor para unos grupos**	Las personas afectadas
Demostrador	Que alguien **pruebe** el modelo	El cliente
Model card	La «etiqueta» del modelo: usos, límites, riesgos	Quien lo adopte
Memoria técnica	Contar el proyecto entero (10-15 páginas)	El cliente y un compañero técnico
Repositorio reproducible	Que **otra persona lo ejecute desde cero**	Cualquiera
Congelación + coevaluación	Cerrar a tiempo y valorar el trabajo en equipo	Evaluación individual

Calendario: día 1 (4/12), **dos días sin clase** (7 y 8/12), día 2 (9/12) y día 3 (10/12, congelación 14:30). Los dos días sin clase son **buenos para avanzar la memoria en equipo**.

1. Evaluación final y análisis de errores

1.1. El test, una sola vez (repaso del Sprint 15)

Evaluáis **el candidato elegido** en el test **una vez**, y anotáis las métricas **con su incertidumbre**. Si no os gusta el resultado, **no se vuelve a elegir mirando el test**: se anota y se explica.

1.2. Gráficos que cuentan algo

Usáis los gráficos de la plantilla `S13_02` (Sprint 13): matriz de confusión, curvas e importancia de variables. Recordad **la regla del Sprint 13**: cada gráfico con **título-conclusión** («Las reservas por móvil se cancelan más»), no con un título descriptivo.

1.3. El análisis de errores: mirar dónde falla

Una métrica global dice **cuánto** falla el modelo, pero no **dónde ni por qué**. El análisis de errores consiste en:

1. **Sacar la lista de los casos donde falla más** (por ejemplo, las 20 reservas con cancelación que el modelo no detectó).
2. **Mirar ejemplos reales** uno a uno.
3. **Buscar qué tienen en común**: ¿son reservas de última hora? ¿un canal concreto? ¿fotos con poca luz?
4. **Anotar lo que se aprende** (alimenta la sección de limitaciones).

> Es la parte más reveladora del proyecto: **de ahí salen las limitaciones honestas** que se piden en la memoria.

2. Sesgos por subgrupos y mitigación

2.1. Qué es un sesgo, en términos prácticos

Un **sesgo** (en el sentido del Sprint 1) = que **el modelo funcione peor para un grupo que para otros**. No hace falta que alguien lo haya querido: sale de los datos (faltan casos de un grupo, hay etiquetas peores…).

Subgrupo = una parte de los datos que tiene sentido separar: por **canal** de reserva, por **isla**, por **tipo de foto** (con sol, nublado), por **tipo de pregunta**, etc.

2.2. Por qué la métrica global puede engañar (ejemplo con números)

Supón un test de **1.000 reservas**: 800 por web y 200 por móvil. Buscamos **recall** (de las que se cancelan, ¿cuántas detectamos?).

Subgrupo	Reservas	Se cancelan	Detectadas	Recall
Web	800	160	128	**80,0 %**
Móvil	200	50	29	**58,0 %**
Total	1.000	210	157	**74,8 %**

- **El número global (74,8 %) parece aceptable**, pero **esconde que en móvil se detecta solo el 58 %**.
- La diferencia es de **22 puntos** entre web y móvil. **Ahí hay un sesgo.**

Una advertencia estadística: el grupo móvil tiene solo 50 cancelaciones, así que **el 58 % tiene mucha incertidumbre** (un intervalo aproximado de **± 14 puntos**: de 44 % a 72 %). En web, con 160, es de **± 6 puntos**. **Con pocos casos hay que ser prudentes al sacar conclusiones** y decir cuánta incertidumbre hay. La diferencia de 22 puntos, aun así, **es mayor que la incertidumbre**, de modo que **merece atención**.

2.3. Qué es una «mitigación» (y por qué hay que medirla)

Una **mitigación** es una **medida para reducir el sesgo**. Opciones habituales (las del reto):

Medida	En qué consiste
Reponderar	Dar **más peso** a los ejemplos del grupo infrarrepresentado al entrenar
Ajustar el umbral por grupo	Decidir «cancelará» con una probabilidad mínima **distinta** para cada grupo
Añadir datos	Conseguir **más ejemplos** del grupo que falta
Limitar el uso	Decir en el aviso: «**no usar** el modelo para el grupo X»

Aplicar una mitigación no basta: hay que medir su efecto, **antes y después, por subgrupo** (es el error frecuente del reto).

2.4. Un ejemplo de mitigación medida (continuación)

Tras reponderar, repetimos la medición:

Subgrupo	Recall antes	Recall después	Cambio
Web	80,0 %	78,8 % (126 de 160)	**−1,2 puntos**
Móvil	58,0 %	68,0 % (34 de 50)	**+10 puntos**
Total	74,8 %	76,2 %	+1,4 puntos

Pero también hay que mirar **el precio que se paga**, por ejemplo en la **precisión** (de las que decimos que se cancelan, ¿cuántas se cancelan de verdad?):

Subgrupo	Precisión antes	Precisión después
Web	76,2 %	73,7 %
Móvil	53,7 %	45,9 %

Lectura honesta: la mitigación **mejora mucho la detección en móvil (+10 puntos)** y **cuesta poco en web (−1,2)**, **pero a cambio hay más falsas alarmas** (la precisión baja). Si cada falsa alarma significa ofrecer un descuento innecesario, **hay que valorar si compensa**. **No existe una mitigación gratis**: la memoria debe **explicar ese equilibrio**.

> **Un buen resultado se parece a esto:** *«Recall de móvil: 58 % → 68 %, sin perjudicar al resto (−1 punto); coste: más falsas alarmas.»* Eso es **medir con honestidad**.

2.5. Si el sesgo no se puede arreglar

A veces **no hay forma de mitigarlo** (faltan datos y no se pueden conseguir). Entonces la respuesta correcta es **documentarlo como limitación**, **avisar en el demostrador** y **limitar el uso**. Esconderlo **no es una opción**.

---

3. Riesgos y normativa: cerrar el registro

El registro de riesgos que empezasteis en el Sprint 14 y revisasteis en el 15 **se cierra aquí**:

- Los **riesgos altos** tienen **estado** (abierto / mitigado / aceptado) y **evidencia** (dónde se ve en el repositorio).
- La **clasificación RGPD / AI Act** (Sprint 1) está **justificada**: ¿riesgo mínimo, limitado o alto? ¿hay datos personales?
- Se indican **supervisión humana** (¿quién revisa las decisiones importantes?) y **transparencia** (¿el usuario sabe que habla con una IA y cuánta confianza tiene el resultado?).

> **Cada afirmación necesita una evidencia comprobable.** «Hemos cuidado la privacidad» no vale; «no hay datos personales, ver `sprint-14/plan_datos_licencias.md`» sí.

---

4. El demostrador

4.1. Qué es (y qué no es)

Un **demostrador** es **una app o cuaderno interactivo donde alguien puede probar el modelo** (Streamlit, Gradio, un cuaderno o una API). **No reentrena**: **carga un modelo ya entrenado** y un archivo `metadatos.json` que le dice qué campos pedir, qué rangos son válidos y con qué datos se entrenó.

4.2. La lógica separada de la interfaz

La estructura que propone la presentación:

```
demostrador/
├── logica.py            # validar + predecir (sin Streamlit)
├── app.py               # interfaz Streamlit: capa fina
├── api.py               # (opcional) FastAPI
├── tests/               # pruebas de sentido común
├── requirements.txt     # versiones fijas
└── README_demostrador.md
```

Por qué se separa: la **lógica** (comprobar entradas y predecir) vive en `logica.py`, **sin nada de interfaz**. Así:

- **se puede probar** con `pytest` sin abrir ninguna ventana;
- **se reutiliza** en el cuaderno, la app y la API;
- si cambiáis la interfaz, **la lógica no se toca**.

Es como un restaurante: **la cocina** (lógica) **y la sala** (interfaz) **son cosas distintas**; puedes cambiar la decoración sin tocar los fogones.

4.3. Tres rasgos de una buena lógica

Rasgo	Ejemplo
Errores en español y accionables	«'noches' debe estar entre 1 y 30 (has escrito -3)», no «ValueError»
Avisa de la incertidumbre	«Riesgo: alto» más un aviso con el AUC del modelo («Estimación orientativa. El modelo tiene un AUC de 0,75…») y sus límites
Devuelve datos, no imprime	Devuelve un diccionario; es la interfaz quien lo muestra

La función `mensaje_errores` del cuaderno (ejercicio 2) transforma una lista de errores en un único texto amable: una cabecera que diga que todavía no se puede calcular y, debajo, una línea por error. Con la lista vacía devuelve un texto vacío. Es una función pequeña que no calcula nada: solo da formato.

4.4. Probar con entradas «malas» a propósito

Un demostrador **debe sobrevivir a un usuario despistado o malintencionado**. Probad como mínimo:

Entrada	Qué debe pasar
3 entradas normales	Resultado coherente
1 entrada imposible (edad 250, fecha en el pasado lejano)	Mensaje claro, **sin romperse**
1 entrada vacía	Mensaje claro

En la review, el docente propondrá **una entrada difícil** para probar **en directo**.

4.5. Pruebas de «sentido común» (`pytest`)

Un modelo **puede fallar sin dar ningún error**: devolver probabilidades absurdas. Las pruebas de sentido común comprueban cosas que **deben cumplirse**:

```
def test_riesgo_alto_puntua_mas():
    alto = predecir({**reserva_ejemplo, "tarifa": "flexible", "deposito": "sin_deposito"})
    bajo = predecir({**reserva_ejemplo, "tarifa": "no_reembolsable", "deposito": "total"})
    assert alto["probabilidad"] > bajo["probabilidad"]
```

Traducción: *«una reserva flexible y sin depósito debe tener más riesgo de cancelación que una no reembolsable con depósito total»*. Si el modelo dice lo contrario, **algo va mal**, aunque no haya ningún error de código. **`pytest -q`** ejecuta todas las pruebas de golpe. El mini-reto pide **3 pruebas propias**.

4.6. El plan B en vídeo

Un **vídeo de 90 segundos** del demostrador funcionando, **por si falla internet** o la app el día de la defensa. Se guarda en `docs/demo.mp4` (o con enlace estable). **Es un seguro barato.**

---

5. El repositorio reproducible

5.1. La prueba definitiva

> **¿Puede otra persona, en otro ordenador, seguir el README y ver todo funcionando?**

Si la respuesta es «solo en mi portátil», **no es reproducible**. Casi siempre falla por **dependencias sin documentar** («en mi ordenador tenía instalada la versión X»).

5.2. Qué debe tener

Pieza	Por qué
`README.md` completo	Problema, resultados, estructura, cómo reproducir, cómo lanzar la demo, datos y licencias, IA responsable, equipo
`requirements.txt` con versiones fijas	Para que se instalen **las mismas versiones** (`pip freeze` recortado a lo que se usa)
Datos o script de descarga	Que otra persona consiga los datos (con la licencia citada)
Script o cuaderno para crear el modelo	Que el README explique **cómo se entrena**
Carpetas ordenadas	`notebooks/`, `src/`, `experimentos/`, `modelo/`, `demostrador/`, `tests/`, `docs/`
Sin claves ni datos personales	Nunca contraseñas, tokens ni datos de personas

5.3. La prueba en limpio

Una persona distinta a la que escribió el README lo sigue en un **Colab nuevo** (o una carpeta limpia): descarga el repositorio como ZIP (**Code → Download ZIP**), abre el cuaderno principal y comprueba que funciona. **Lo que falle, se corrige.** Si lo prueba quien lo escribió, **no vale**: ya sabe «los trucos» que no están anotados.

---

6. La model card

6.1. Qué es

La **model card** es **la «etiqueta nutricional» del modelo**: una ficha de **máximo 2 páginas** que dice **para qué sirve, con qué datos se entrenó, cuánto vale y cuándo NO usarlo**. Se guarda como `docs/model_card.md` y **debe coincidir con la memoria y con el aviso del demostrador**.

6.2. Las nueve partes (de la plantilla)

Parte	Qué contiene
1. Detalles	Nombre, versión, tipo de modelo, librería con versión, semilla, licencia
2. Uso previsto	Para qué, para quién, **qué decisión sigue siendo humana**, **usos NO permitidos**
3. Datos	Fuente, licencia, tamaño, cómo se partió, sesgos de muestreo
4. Factores relevantes	Grupos o condiciones que cambian el rendimiento (isla, iluminación…)
5. Métricas	Baseline frente al modelo, con incertidumbre
6. Subgrupos	Rendimiento por grupo (la tabla del apartado 2)
7. Ética y riesgos	AI Act / RGPD, mitigaciones, supervisión humana
8. Limitaciones	**Cuándo no fiarse** y qué haría falta antes de un despliegue real
9. Contacto	Responsable y cada cuánto se reevalúa

> **La model card es la parte donde más cuenta la honestidad.** Un modelo con límites claros es **más fiable** que uno que dice valer para todo.

7. La memoria técnica (10-15 páginas)

7.1. Para quién se escribe

Para **el cliente y un compañero técnico**: **autocontenida** (que se entienda sin haber seguido el proyecto) y **propia** (no copiada). Si usáis IA para redactar, **revisadla y declaradlo** (y **comprobad que no ha cambiado ninguna cifra**).

7.2. Estructura y presupuesto de páginas

Sección	Páginas aprox.	Requisitos
0. Resumen ejecutivo	½-1 (**se escribe al final**)	R1, R10
1. Problema y valor de negocio	1	R1
2. Datos y EDA	2	R2
3. Metodología	1	R3
4. Modelado y experimentos	2	R3, R4
5. Evaluación y resultados	2	R5
6. IA responsable	2	R6
7. Demostrador y reproducibilidad	1	R7, R8
8. Gestión del proyecto	½-1	R9
9. Conclusiones y recomendaciones	1	

Con el resumen y la gestión en su máximo, la suma es de 14 páginas, dentro del límite de 15. (Portada, índice, referencias y **anexos no cuentan**.)

7.3. Cómo escribirla sin sufrir

1. **Empezad por el esqueleto** (`plantilla_esqueleto_memoria_tecnica.md`): ya tiene secciones y un mapa de requisitos R1-R11.
2. **Cada sección sale de algo que ya existe** (la ficha del Sprint 14, el registro del 15, la evaluación del 16): **se trata de contar, no de inventar**.
3. **Borrador el 9/12, revisión el 10/12.** El error clásico es dejarla para el último día.
4. **Cifras coherentes** entre memoria, README, model card y registro. **Si una cifra aparece en dos sitios, tiene que coincidir.**
5. **El resumen se escribe al final**, con **una recomendación al cliente y un límite clave**.

---

8. Congelación y coevaluación

8.1. La congelación

A las 14:30 del 10/12 se evalúa lo que haya en la **rama principal**. Para dejarlo claro se crea una **etiqueta** (*tag*) en Git llamada **`v1.0-entrega` antes de las 14:30**. Después **no se toca código ni memoria** (solo la presentación y el vídeo). Un fallo crítico del demostrador solo se corrige **con permiso del docente** y dejándolo documentado.

El `checklist_congelacion.md` tiene seis bloques: **A** repositorio, **B** reproducibilidad (en limpio), **C** demostrador, **D** evaluación e IA responsable, **E** memoria y **F** equipo y metodología. **Cada casilla sin marcar es un riesgo.**

8.2. La coevaluación

Cada integrante rellena **la coevaluación individual** (anexo B): valora la contribución de cada persona del equipo. Junto con **el historial de commits**, **ajusta la nota individual con un factor de 0,8 a 1,1**.

Situación	Efecto
Aporte real y repartido	Factor ≈ 1 (o algo más)
Aporte muy bajo (poco o ningún commit)	Factor hacia 0,8
No entregar la coevaluación	Se pierde un elemento de la nota individual

> **La coevaluación se entrega aunque sea incómoda**: es una herramienta de **justicia dentro del equipo**, no una denuncia.

---

9. Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Memoria el último día	«Total, ya está hecho»	Borrador el 9/12, revisión el 10/12
Demostrador que solo funciona en un ordenador	Dependencias sin documentar	`requirements.txt` con versiones y prueba en limpio
Mitigación aplicada pero no medida	Se cree que basta con hacerla	Métricas **antes y después, por subgrupo**
Subir cambios tras las 14:30	Se sigue retocando	Crear la etiqueta antes y no tocar
Memoria > 15 páginas o con texto copiado	Se llena de relleno	Presupuesto de páginas; anexos aparte
Cifras distintas en memoria y README	Se copian de versiones distintas	Una revisión final de coherencia
Solo el responsable técnico hace commits	El resto «no sabe Git»	Reparto real; vale hacer un cambio pequeño cada uno
Coevaluación sin entregar	Se olvida	Cada persona la entrega

---

10. Dónde practicar cada cosa

Si no entiendes...	Mira la sección	Practica en
Evaluación final y errores	1	Paso 1 del reto (cuaderno 04, plantilla `S13_02`)

Si no entiendes...	Mira la sección	Practica en
Sesgos por subgrupo y mitigación	2	Tutoría de sesgos; «Mitigar sesgos y volver a medir»
Riesgos y AI Act	3	`docs/riesgos_eticos.md`
Demostrador, validación y pruebas	4	Cuaderno `S16_01_demostrador`; carpeta `demostrador/`
README y reproducibilidad	5	Checklist de congelación, bloques A y B
Model card	6	`plantilla_model_card.md`
Memoria técnica	7	`plantilla_esqueleto_memoria_tecnica.md`
Congelación y coevaluación	8	`checklist_congelacion.md`, `plantilla_coevaluacion.md`

11. Chuleta de vocabulario

Palabra	En lengua normal
Congelación	Momento a partir del cual no se cambia nada
Etiqueta (tag)	Marca de Git que señala la versión entregada
Análisis de errores	Estudiar dónde y por qué falla el modelo
Subgrupo	Parte de los datos que se mide por separado
Sesgo	Que el modelo funcione peor para un grupo
Mitigación	Medida para reducirlo, que hay que medir
Demostrador	App o cuaderno donde se prueba el modelo
Plan B	Vídeo por si falla la demo
Model card	Ficha técnica del modelo
Memoria técnica	Documento de 10-15 páginas del proyecto
Reproducible	Otra persona puede ejecutarlo desde cero
`pytest`	Herramienta que ejecuta las pruebas automáticas
Coevaluación	Valoración individual entre compañeros

12. Cómo estudiar estos apuntes

1. **Antes del día 1**, leed las secciones 1 y 2: guían la evaluación y los sesgos.
2. **Antes de montar el demostrador**, leed la sección 4; **antes de redactar**, las 6 y 7.
3. **La víspera de la congelación**, repasad la sección 8 y pasad el `checklist_congelacion.md` con alguien que no haya escrito el README.
4. **Regla final:** si alguien ajeno **no puede clonar el repositorio, seguir el README y ver la demo funcionando**, **el proyecto no está entregado**, por bueno que sea el modelo.